

NIS2 Uyumluluğu için Yazılım Geliştirme ve Kanıt Rehberi

Proje: GD-PMS (Power Management System)

Mimari: Nx Monorepo (Backend Servisler + Desktop App + React Frontend + Paylaşılan Paketler)

Dil: TypeScript

Monorepo Adı: gd-pms-monorepo

Versiyon: 1.3

Bu belge, NIS2 Direktifi kapsamında Nx monorepo yapısındaki GD-PMS yazılımının geliştirme sürecinde alınması gereken teknik ve süreçsel önlemleri tanımlamaktadır. Mevcut durumda SonarQube entegrasyonu tamamlanmıştır.

Mevcut Durum ve Mimari Özet

Bileşen	Teknoloji	Durum
Monorepo Yönetimi	Nx	Kurulu
Paket Yöneticisi	Bun	Kurulu
Backend Servisler	Node.js (packages/services/)	Geliştirme süreci devam ediyor
Desktop Uygulama	Electron/Container (apps/desktop, apps/container-desktop)	Geliştirme süreci devam ediyor
Web Frontend	React (apps/container-web, apps/field, apps/superadmin)	Geliştirme süreci devam ediyor
Paylaşılan Paketler	core, shared-types, shared-utils, ui, device-library	Mevcut
Kod Kalitesi	SonarQube	Entegre

OWASP Standartları	Kod seviyesinde uygulandı	Uygulandı
Test Çerçevesi	Vitest	Mevcut
E2E Test	Playwright	Mevcut

Faz 1: SonarQube Sertleştirme

Adım 1: NIS2'ye Özel Kalite Profili (Quality Profile) Oluşturulması

Amaç: TypeScript/JavaScript için özel bir güvenlik profilinin uygulandığının denetçiye gösterilmesi.

Yapılacaklar:

1. SonarQube arayüzünde **Quality Profiles** bölümüne gidilir.
2. TypeScript için mevcut varsayılan profil (Sonar way recommended) kopyalanarak **"NIS2 Compliance - TypeScript"** adında yeni bir profil oluşturulur.
3. Bu profile OWASP Top 10, SANS Top 25 ve "Hard-coded credentials" dahil olmak üzere yerleşik güvenlik kural setleri eklenir.
4. Aynı profilin **JavaScript** için de oluşturulması önerilir (bazı yapılandırma dosyaları JS formatında olabilir).

Kanıt: SonarQube profil yapılandırması ekran görüntüsü.

Adım 2: Tamper-Evident Loglama için Özel Kural Yazılması

Amaç: Geliştiricilerin standart ve güvensiz loglama yöntemlerini kullanmasının engellenmesi.

Yapılacaklar:

1. **Custom Rules** bölümünden TypeScript için yeni bir kural oluşturulur.
2. Şiddet seviyesi **Blocker** olarak belirlenir.
3. Kurum bünyesinde geliştirilen `TamperEvidentLogger` (packages/core içerisinde yer alacaktır) haricindeki tüm ham log çağrılarını tespit eden XPath kuralı yazılır.
4. Özellikle `console.log`, `console.error`, `console.warn` ve backend servislerdeki doğrudan logger çağrıları hedef alınır.
5. Hata mesajı şu şekilde tanımlanır: "NIS2 Madde 21(b): Tamper-evident loglama zorunludur. Lütfen '@gd-pms/core' içindeki 'TamperEvidentLogger'ı kullanın."

6. Bu kural “NIS2 Compliance - TypeScript” profiline eklenir.

Monorepo Notu: Bu kural tüm apps ve packages altındaki projelerde geçerli olacaktır. Frontend uygulamaları (container-web, field, superadmin) ve desktop uygulamaları için tamper-evident loglama yerine audit log mekanizması değerlendirilmelidir. Bu projeler için ayrı bir profil oluşturulup kural “Major” seviyesine çekilebilir.

Kanıt: Kural tanımının ekran görüntüsü ve ihlal durumunda oluşan hata mesajı.

Adım 3: Hard-Coded Secret ve Hassas Veri Kurallarının Aktif Edilmesi

Amaç: API anahtarı, şifre ve token gibi hassas bilgilerin kaynak kod içerisinde gömülü olarak bulunmasının engellenmesi.

Yapılacaklar:

1. “NIS2 Compliance - TypeScript” profili içerisinde “Hard-coded credentials”, “Hard-coded secrets” ve “Potential sensitive data exposure” kategorilerindeki tüm kurallar tespit edilir.
2. Tespit edilen kuralların tamamı **Blocker** seviyesinde aktifleştirilir.
3. `.env.example` ve test dosyalarının bu kurallara takılmaması için SonarQube exclusion list'ine ilgili dosyalar eklenir. Mevcut `sonar-project.properties` dosyası güncellenir.

Kanıt: İlgili kuralların “Blocker” ve “Active” durumunda olduğunu gösteren ekran görüntüsü.

Adım 4: Kalite Kapısının (Quality Gate) Monorepo Yapısına Göre Sertleştirilmesi

Amaç: Belirlenen güvenlik eşiklerini geçemeyen kodun canlı ortama çıkmasının engellenmesi.

Yapılacaklar:

1. **Quality Gates** ayarlarına gidilir.
2. “NIS2 Gate - GD-PMS” adında yeni bir kapı oluşturulur.
3. Aşağıdaki koşullar eklenir (tamamı “Yeni Kod” için geçerlidir):
 - Blocker Issues > 0 ise: **Başarısız**
 - Critical Issues > 0 ise: **Başarısız**
 - Security Rating < A ise: **Başarısız**

- Coverage on New Code < %60 ise: **Başarısız**
4. sonar-project.properties dosyasında monorepo yapısına uygun kaynak ve test dizinleri tanımlanır. Mevcut dosya aşağıdaki gibi güncellenir:
 - sonar.sources değerine apps ve packages altındaki tüm src dizinleri eklenir.
 - sonar.tests değerine tüm spec ve test dizinleri eklenir.
 - sonar.exclusions değerine e2e, configs ve deployment dizinleri eklenir.
 5. Nx projesinde her bir app ve package için ayrı SonarQube projesi oluşturulması önerilir.

Monorepo Notu: nx.json ve her bir projenin project.json dosyasında SonarQube hedefi tanımlanabilir. Bu yöntem, hangi projenin Quality Gate'i geçemediğinin net olarak görülmesini sağlar.

Kanıt: Quality Gate konfigürasyonu ekran görüntüsü, güncellenmiş sonar-project.properties dosyası ve başarılı/başarısız bir Nx build sonucu.

Faz 2: Kod İçi Güvenlik Mimarisi

Adım 5: packages/core İçerisinde TamperEvidentLogger Sınıfının Yazılması

Amaç: Backend servisler (data-service, device-service, web-service) tarafında NIS2 olay yönetimi (Madde 21-b) gerekliliklerini karşılayan, değiştirilemez logların üretilmesi.

Yapılacaklar:

1. packages/core/src/lib/ altında logger/ dizini oluşturulur.
2. Bu dizine TamperEvidentLogger sınıfı yazılır. Bu sınıf, servislerin mevcut logger altyapısını sarmalayacak (wrap) şekilde tasarlanır.
3. **Teknik Gereksinimler:**
 - Her log satırına **sıralı ID, ISO 8601 timestamp, bir önceki satırın SHA-256 hash'i, log seviyesi, mesaj ve metadata** eklenir.
 - SIEM sistemlerinin doğrudan parse edebilmesi için log formatı olarak **JSON** çıktısı üretilir.
 - Admin girişi, cihaz konfigürasyon değişikliği, setpoint güncellemesi gibi kritik işlemler **audit** seviyesinde özel olarak işaretlenir.
4. Bu kütüphane packages/core/src/index.ts üzerinden export edilerek tüm servisler ve uygulamalar tarafından kullanılabilir hale getirilir.

Monorepo Notu: packages/core hem backend servisler hem desktop uygulamalar hem de frontend uygulamaları tarafından import edilebilir. Asıl tamper-evident loglama

backend servislerde çalışacaktır. Desktop ve web uygulamaları için daha hafif bir audit log mekanizması `packages/core` içerisinde ayrı bir modül olarak sunulabilir.

Kanıt: Kaynak kod (`packages/core/src/lib/logger/tamper-evident-logger.ts`) ve üretilen örnek log çıktısı.

Adım 6: Backend Servislere Logger Plugin Entegrasyonu

Amaç: TamperEvidentLogger'ın tüm backend servislerin request/response döngüsüne dahil edilmesi ve otomatik audit log üretilmesi.

Yapılacaklar:

1. `packages/services/data-service/src/` , `packages/services/device-service/src/` ve `packages/services/web-service/src/` altında logger entegrasyonu gerçekleştirilir.
2. Her servis için ortak bir logger yapılandırması `packages/core` içerisinden sağlanır.
3. Hassas endpoint'lere yapılan istekler otomatik olarak `audit` seviyesinde loglanır.
4. Response süresi ve status code loga eklenir.
5. Her servis kendi logger konfigürasyonunu `packages/core` üzerinden alacak şekilde yapılandırılır.

Monorepo Notu: Tüm backend servisler `packages/core` kütüphanesindeki TamperEvidentLogger'ı import edecektir. Bu sayede monorepo içerisinde tek bir loglama kaynağı bulunacak ve log formatı tüm servislerde tutarlı olacaktır.

Kanıt: Logger yapılandırma dosyaları ve örnek audit log çıktısı.

Adım 7: VerifyChain() Metodunun ve Testinin Yazılması

Amaç: Log bütünlüğünün otomatik testinin yapılarak teknik kanıt üretilmesi.

Yapılacaklar:

1. TamperEvidentLogger sınıfına, log dosyasını satır satır okuyup her satırın `previousHash` alanını bir sonraki satırın hash'i ile karşılaştıran statik bir metod eklenir.
2. `packages/core/src/lib/logger/tamper-evident-logger.spec.ts` dosyasında aşağıdaki senaryoyu kapsayan bir test yazılır:
 - 10 adet log üretilir (geçici bir dosyaya yazılır).
 - `VerifyChain` metodu çağrılır ve `true` döndüğü doğrulanır (assert).
 - Test ortamında log dosyasının 5. satırındaki mesaj manuel olarak değiştirilir.
 - `VerifyChain` metodu tekrar çağrılır ve `false` döndüğü doğrulanır (assert).

3. Bu test `@nis2-security` tag'i ile işaretlenir.

Kanıt: Test kodu dosyası ve başarılı CI test sonucu.

Adım 8: Health Check Endpoint'inin Yazılması

Amaç: İzleme sistemlerinin backend servislerin güvenlik duruşunu anlık olarak okuyabilmesinin sağlanması.

Yapılacaklar:

- Her backend serviste (data-service, device-service, web-service) bir health check endpoint'i tanımlanır.
- `GET /api/nis2/health` endpoint'i aşağıdaki bilgileri içeren bir JSON döndürür:
 - `status` : "healthy" veya "degraded"
 - `log_chain_intact` : true/false (son log dosyası için VerifyChain sonucu)
 - `db_connection` : "ok" veya "error"
 - `last_audit_entry` : son audit log'un timestamp'i
 - `uptime` : process.uptime()
- Endpoint'in kötüye kullanımını önlemek amacıyla rate limiting uygulanır.

Kanıt: Endpoint'e yapılan bir isteğin cevabının ekran görüntüsü.

Adım 9: Frontend ve Desktop Uygulamalar için Audit Log Hook'unun Yazılması

Amaç: Kullanıcı arayüzünde gerçekleşen kritik kullanıcı işlemlerinin backend servislere audit log olarak iletilmesi.

Yapılacaklar:

- `packages/core/src/lib/logger/` altında `useAuditLog` adında bir hook oluşturulur.
- Bu hook, kritik kullanıcı işlemlerini (buton tıklama, form gönderimi, cihaz yapılandırması) ilgili backend servisine iletir.
- Gönderilen veri: `action` , `userId` , `timestamp` , `applicationName` (container-web/desktop/field/superadmin), `componentName` , `metadata` .
- Backend servislerde bu endpoint'e gelen veriler `TamperEvidentLogger` ile audit seviyesinde loglanır.
- Hook aşağıdaki uygulamalarda kullanıma sunulur:
 - `apps/container-web`
 - `apps/field`

- apps/superadmin
- apps/desktop
- apps/container-desktop

Monorepo Notu: Hook, packages/core içerisinde tutulduğundan tüm frontend ve desktop uygulamaları tarafından ortak şekilde kullanılabilir. packages/ui içerisindeki bileşenler de bu hook'u kullanarak audit log üretebilir.

Kanıt: Hook kaynak kodu ve backend'de üretilen audit log örneği.

Adım 10: Cihaz Simülatörleri için Güvenlik Entegrasyonu

Amaç: packages/simulators içerisindeki cihaz simülatörlerinin ürettiği logların da NIS2 uyumlu hale getirilmesi.

Yapılacaklar:

1. Simülatörlerin kritik işlemleri (cihaz bağlantısı, veri gönderimi, konfigürasyon değişikliği) TamperEvidentLogger ile loglanır.
2. Test ortamlarında simülatör loglarının bütünlüğü VerifyChain ile doğrulanır.

Kanıt: Simülatör log çıktısı ve doğrulama sonucu.

Faz 3: CI/CD Kanıt Fabrikası

Adım 11: Nx ile SonarQube Analizinin Zorunlu Hale Getirilmesi

Amaç: Kalite kapısından geçemeyen kodun merge edilmesinin engellenmesi.

Yapılacaklar:

1. CI/CD pipeline konfigürasyonunda nx affected:lint ve nx affected:test adımlarının ardından SonarQube taraması çalıştırılır.
2. Tarama adımına sonar.qualitygate.wait=true parametresi eklenir.
3. Nx'in affected komutları kullanılarak yalnızca değişen projeler taranır.
4. Mevcut sonar-project.properties dosyasındaki yapılandırma CI ortamı için optimize edilir.

Kanıt: Başarısız bir SonarQube kontrolü sonrası duran Nx build'inin logu.

Adım 12: Güvenlik Testlerinin Ayrı Bir Nx Hedefi Olarak Tanımlanması

Amaç: NIS2'ye özel testlerin pipeline'da ayrıştırılmasının ve raporlanmasının sağlanması.

Yapılacaklar:

1. `packages/core/project.json` dosyasında `test-nis2` adında yeni bir test hedefi tanımlanır.
2. Bu hedef, yalnızca `@nis2-security` tag'ine sahip testleri çalıştıracak şekilde yapılandırılır.
3. CI pipeline'ında `nx test-nis2 core` komutu ayrı bir adım olarak çalıştırılır.
4. Bu adımın çıktısı özel bir artifact olarak saklanır.
5. Aynı hedef diğer kritik paketler (services, shared-types) için de tanımlanır.

Kanıt: CI logunda `@nis2-security` testlerinin çalıştığını gösteren bölüm.

Adım 13: SBOM Üretiminin ve Taramasının Otomatikleştirilmesi

Amaç: Proje için tedarik zinciri güvenliğinin (NIS2 Madde 21-d) kanıtlanması.

Yapılacaklar:

1. Monorepo root'una `@cyclonedx/bom` eklentisi eklenir.
2. CI pipeline'ına her build sonunda `sbom.json` dosyası üretecek bir adım eklenir. Bu adım tüm monoreponun bağımlılıklarını tarar.
3. Üretilen SBOM dosyası **Trivy** veya **Dependency-Track** ile taranır. Kritik bir CVE tespit edilmesi durumunda build "Failed" durumuna geçer.
4. Bun paket yöneticisi kullanıldığı için `bun.lock` dosyası bağımlılık analizine dahil edilir.

Monorepo Notu: apps ve packages altındaki her projenin kendi `package.json` bağımlılıkları bulunmaktadır. CycloneDX tümünü tek bir SBOM'da birleştirecektir.

Kanıt: SBOM dosyası ve "0 Critical/High CVE" içeren tarama raporu.

Adım 14: E2E Güvenlik Testlerinin Playwright'a Entegrasyonu

Amaç: Uçtan uca güvenlik senaryolarının otomatik testinin yapılması.

Yapılacaklar:

1. Mevcut `playwright.config.ts` dosyasına güvenlik testleri için özel bir proje tanımı eklenir.
2. `e2e/` dizini altında `security/` klasörü oluşturulur.
3. Aşağıdaki senaryoları kapsayan testler yazılır:
 - Yetkisiz erişim girişimleri
 - Audit log üretiminin doğrulanması
 - Health check endpoint'lerinin çalışırılığı
 - Kritik işlemlerin loglanması
4. Bu testler `@nis2-security` tag'i ile işaretlenir.

Kanıt: Playwright güvenlik testleri çalıştırma sonucu.

Adım 15: Otomatik Sürüm Güvenlik Beyanının Oluşturulması

Amaç: Her sürüm için NIS2 uygunluk kanıtlarını birleştiren imzalı bir belgenin otomatik olarak üretilmesi.

Yapılacaklar:

1. Pipeline'ın son adımında çalışacak bir script yazılır (TypeScript ile yazılıp Bun ile çalıştırılabilir).
2. Script aşağıdaki işlemleri gerçekleştirir:
 - **SonarQube API** üzerinden Quality Gate sonucunu çeker.
 - **Vitest JUnit XML** çıktılarından geçen/kalan sayısını ve `@nis2-security` testlerinin başarı durumunu parse eder.
 - **Playwright** raporundan güvenlik testi sonuçlarını okur.
 - **Trivy** raporundan kritik CVE sayısını okur.
 - Tüm bu verileri önceden hazırlanmış bir HTML şablonuna basarak `release-sec-evidence-vX.Y.Z.html` dosyasını oluşturur.
3. Belgenin altına "Bu sürüm, NIS2 Madde 21 teknik kontrollerinden otomatik olarak geçmiştir" ibaresi eklenir.
4. İsteğe bağlı olarak belge PGP/GPG ile imzalanır.

Monorepo Notu: Script `scripts/` veya `tools/` altında tutulabilir ve `bun run` ile çalıştırılabilir. `nx.json` içerisine bu script için bir runner tanımı eklenebilir.

Kanıt: Otomatik oluşturulmuş bir Sürüm Güvenlik Beyanı örneği.

Faz 4: Manuel ve Yönetimsel Hazırlıklar

Adım 16: Güvenli Geliştirme Politikası (SSDF) Belgesinin Hazırlanması

Amaç: NIS2 yönetimsel yükümlülüklerinin karşılanması.

İçerik: Belge aşağıdaki maddeleri kapsamalıdır:

- Yeni bir servis, uygulama veya bileşen eklenmeden önce tehdit modellemesi yapılır.
- Tüm kod değişiklikleri SonarQube "NIS2 Compliance - TypeScript" profilinden geçer.
- Tamper-evident loglama (Madde 21-b) zorunlu olup @gd-pms/core içerisindeki logger tüm backend servislerde kullanılır.
- Tüm kullanıcı arayüzlerindeki (container-web, field, superadmin, desktop, container-desktop) kritik işlemler audit log olarak backend'e gönderilir.
- Cihaz simülatörleri de dahil olmak üzere tüm bileşenler NIS2 loglama standartlarına uyar.
- Bağımlılıklar her build'de taranır ve kritik zafiyetlere izin verilmez (SBOM + Trivy).
- Yılda en az bir kez harici bağımsız sızma testi yaptırılır (tüm servisler ve uygulamalar ayrı ayrı test edilir).

Onay: Şirket üst yönetimi tarafından imzalanmalıdır.

Kanıt: İmzalı PDF belge.

Adım 17: Harici Bağımsız Sızma Testinin (Pentest) Planlanması

Amaç: Kurum içi testlerin haricinde bağımsız bir doğrulama sağlanması.

Yapılacaklar:

1. Yılda en az bir kez, akredite bir firma tarafından tüm sistem bileşenleri için **OWASP metodolojisine uygun** sızma testi yaptırılır.
2. Test kapsamına aşağıdakiler dahil edilir:
 - Backend servisler (data-service, device-service, web-service) için OWASP API Security Top 10
 - Web uygulamaları (container-web, field, superadmin) için OWASP Top 10
 - Desktop uygulamaları (desktop, container-desktop) için masaüstü güvenlik kontrolleri
 - Servisler arası iletişim güvenliği
3. Test raporu ve bulguların kapatıldığına dair kanıtlar saklanır.

Kanıt: Harici pentest raporu ve iyileştirme takip çizelgesi.

Monorepo Dizin Planı

Bu adımların uygulanması sonucunda workspace'te aşağıdaki ekleme ve değişiklikler yapılacaktır:

Yeni Eklenecek Dosyalar:

- packages/core/src/lib/logger/tamper-evident-logger.ts
- packages/core/src/lib/logger/tamper-evident-logger.spec.ts
- packages/core/src/lib/logger/useAuditLog.ts
- packages/core/src/lib/logger/index.ts
- scripts/generate-release-evidence.ts

Güncellenecek Dosyalar:

- sonar-project.properties (NIS2 exclusion ve kaynak dizinleri)
- nx.json (yeni runner tanımları)
- playwright.config.ts (güvenlik testi projesi)
- packages/services/data-service/src/ (logger entegrasyonu)
- packages/services/device-service/src/ (logger entegrasyonu)
- packages/services/web-service/src/ (logger entegrasyonu)
- packages/simulators/ (logger entegrasyonu)

Özet Kontrol Listesi

#	Adım	Kategori	Konum
1	TypeScript için NIS2 Quality Profile oluşturulması	SonarQube	SonarQube UI
2	Tamper-evident loglama Blocker kuralının yazılması	SonarQube	SonarQube UI
3	Hard-coded secret kurallarının aktif edilmesi	SonarQube	SonarQube UI
4	Quality Gate'in sertleştirilmesi ve sonar-project.properties güncellemesi	SonarQube	SonarQube UI / Root
5	TamperEvidentLogger sınıfının yazılması	Kod	packages/core/src/lib/logger/

6	Backend servislere logger entegrasyonu	Kod	packages/services/*/
7	VerifyChain metodu ve testinin yazılması	Kod / Test	packages/core/src/lib/logger/
8	Health check endpoint'inin yazılması	Kod	packages/services/*/
9	useAuditLog hook'unun yazılması ve uygulamalara entegrasyonu	Kod	packages/core + apps/*/
10	Simülatörlere güvenlik entegrasyonu	Kod	packages/simulators/
11	Nx ile SonarQube analizinin CI'a eklenmesi	CI/CD	CI konfigürasyonu
12	@nis2-security testlerinin ayrı Nx hedefi yapılması	CI/CD	packages/*/project.json
13	SBOM üretilmesi ve Trivy ile taranması	CI/CD	CI konfigürasyonu
14	Playwright güvenlik testlerinin eklenmesi	CI/CD	e2e/security/
15	Otomatik Sürüm Güvenlik Beyanı oluşturulması	CI/CD	scripts/
16	SSDF belgesinin hazırlanması ve imzalatılması	Doküman	docs/
17	Yıllık bağımsız pentest planlanması	Doküman	Dış kaynak

Ek A: NIS2 Uyumluluk Denetimi ve Sertifikasyon Kuruluşları

NIS2'de Resmi Sertifikasyon Durumu

NIS2 Direktifi, GDPR'da olduğu gibi resmi bir sertifikasyon programına sahip değildir. "NIS2 Approved" etiketi basan merkezi bir otorite veya standart bir logo mevcut değildir. Avrupa Komisyonu, NIS2 kapsamında Avrupa Siber Güvenlik Sertifikasyon Çerçevesi

(EUCC) üzerinde çalışmalarını sürdürmektedir; ancak bu çalışma henüz NIS2'ye özel bir sertifikaya dönüşmemiştir.

Mevcut durumda NIS2 uyumluluğu, bir denetim (audit) ve uygunluk değerlendirme (conformity assessment) süreci olarak yürütülmektedir. Kuruluşlar, akredite denetim firmalarından "NIS2 Uygunluk Raporu" (NIS2 Compliance Report) almaktadır.

Hizmet Veren Kuruluş Tipleri

1. Büyük Dörtlü (Big Four) Danışmanlık ve Denetim Firmaları

Kuruluşlar: Deloitte, PwC, EY, KPMG

Hizmet Kapsamı:

- NIS2 boşluk analizi (gap analysis)
- Uyumluluk yol haritası oluşturma
- Resmi uygunluk raporu düzenleme
- Ağırlıklı olarak büyük kurumsal şirketlerle çalışma

Avantajları: Ulusal otoriteler nezdinde en yüksek güvenilirlik seviyesi; raporları sorgulanmamaktadır.

Dezavantajları: Yüksek maliyet (50.000 Euro'dan başlamakta, proje büyüklüğüne bağlı olarak 200.000-300.000 Euro seviyesine ulaşabilmektedir).

2. Butik Siber Güvenlik Danışmanlık Firmaları

Kuruluşlar: WithSecure, NCC Group, S-RM, NVISO, SEC Consult (Türkiye'de Biznet Bilişim, BGA Security, Picus Security)

Hizmet Kapsamı:

- Teknik pentest ve NIS2 danışmanlığı
- ISO 27001 ile NIS2'nin harmanlanarak entegre uyumluluk paketi sunulması
- Daha esnek ve hızlı hizmet

Avantajları: Big Four'a kıyasla daha uygun maliyet (10.000 - 50.000 Euro aralığı); teknik detaylara daha yüksek hakimiyet.

Dezavantajları: Bazı ülke otoriteleri tarafından daha az tanınabilmekte; raporun kabulü ülkeye göre değişiklik gösterebilmektedir.

3. GRC (Governance, Risk, Compliance) Yazılım Firmaları

Kuruluşlar: Vanta, Drata, OneTrust, Scrut, Secureframe

Hizmet Kapsamı:

- Danışmanlıktan ziyade yazılım satışı
- Sürekli uyumluluk izleme (continuous compliance)
- Otomatik denetim kanıtı toplama
- Bazıları (Vanta, Drata) harici denetçi bağlantısı ile resmi rapor alınmasını da sağlamaktadır

Avantajları: Otomasyon odaklı yaklaşım; yıllık abonelik modeli (5.000 - 20.000 USD/yıl).

Dezavantajları: Tamamen yazılıma dayalı güvencenin yetersizliği; insan denetçi zorunluluğu devam etmektedir. Daha çok bulut-yerel startuplar için uygundur.

Yetkili Ulusal Otoriteler

NIS2 Direktifi, her AB üyesi ülkenin kendi yetkili ulusal otoritesini (NCA - National Competent Authority) belirlemesini zorunlu kılmaktadır. GD-PMS yazılımının satışının yapılacağı ülkeye göre muhatap kurum değişiklik göstermektedir:

Ülke	Yetkili Otorite
Almanya	BSI (Bundesamt für Sicherheit in der Informationstechnik)
Fransa	ANSSI (Agence Nationale de la Sécurité des Systèmes d'Information)
Hollanda	NCSC (Nationaal Cyber Security Centrum)
İspanya	INCIBE (Instituto Nacional de Ciberseguridad)
İtalya	ACN (Agenzia per la Cybersicurezza Nazionale)
Belçika	CCB (Centre for Cybersecurity Belgium)

Bu otoriteler denetimi doğrudan gerçekleştirmemektedir. Ancak hangi denetim firmalarının raporlarının kabul edileceğini belirleme yetkisine sahiptirler. Bazı ülkeler yalnızca akredite firmaların raporlarını tanırken, bazıları daha esnek bir yaklaşım benimsemektedir.

NIS2 Uyumluluk Yol Haritası

- Hedef Pazarın Belirlenmesi:** GD-PMS yazılımının satılacağı AB ülkesi tespit edilir. İlgili ülkenin NCA web sitesinden "NIS2 conformity assessment body" veya "accredited auditor" listeleri incelenir.
- Ön Denetim (Gap Analysis) Yaptırılması:** Bir danışmanlık firması ile anlaşarak mevcut durum analizi gerçekleştirilir. Teknik altyapı (SonarQube, pipeline) güçlü

çıkacak olup eksiklerin genellikle yönetsel alanlarda (politikalar, tatbikat kayıtları, tedarikçi sözleşmeleri) yoğunlaşması beklenmektedir.

3. **Eksiklerin Kapatılması:** Gap analysis raporunda belirtilen bulgular giderilir.
4. **Resmi Denetim (Audit) Alınması:** Aynı firma veya farklı bir akredite firma tarafından tam denetim gerçekleştirilir. Başarılı sonuç durumunda "NIS2 Conformity Statement" veya "NIS2 Compliance Report" düzenlenir.
5. **Belgenin Muhafazası ve Güncellenmesi:** Alınan belge satış sözleşmelerine ek olarak sunulabilir. Her büyük sürüm sonrası veya yıllık periyotlarla yenilenmesi gerekebilir.

Türkiye Açısından Değerlendirme

Türkiye AB üyesi olmadığından NIS2 Direktifi doğrudan bağlayıcılık taşımamaktadır. Bununla birlikte, hedef müşteri kitlesi AB'deki enerji şirketleri olduğunda, bu şirketlerin NIS2 uyumluluğu tedarikçi olarak kurumu da kapsayacaktır. Bu bağlamda:

- Müşteriler "NIS2 uygunluk kanıtı" talep edecektir.
- Türkiye'deki danışmanlık firmaları NIS2 konusunda yardımcı olabilmekle birlikte, AB otoriteleri nezdinde geçerli rapor için AB merkezli bir firma ile çalışılması daha güvenceli bir yaklaşımdır.
- Bazı Türk firmaları, AB'deki iş ortakları aracılığıyla ortak denetim gerçekleştirebilmektedir.

Ek B: OSCP Sertifikası ve Pentest Durumu

OSCP ile Kurum İçi Pentest Yapılmasının NIS2 Açısından Değerlendirilmesi

OSCP (Offensive Security Certified Professional), pratik sızma testi becerilerini kanıtlayan, uluslararası düzeyde en saygın sertifikalardan biridir. OSCP sertifikasına sahip personelin teknik olarak pentest gerçekleştirme yetkinliği bulunmaktadır. NIS2 açısından durum aşağıdaki şekilde değerlendirilmektedir:

OSCP'nin Sağladığı Avantajlar

- **Sürekli İç Güvenlik Testleri:** OSCP yetkinliği ile her sprint sonunda veya aylık periyotlarla tüm servisler (data-service, device-service, web-service) ve uygulamalar (container-web, field, superadmin, desktop, container-desktop) üzerinde iç pentest gerçekleştirilebilir. Bu uygulama, NIS2'nin "sürekli iyileştirme" maddesi için güçlü bir kanıt teşkil eder.

- **Güvenli Kod Yazma Refleksi:** Saldırgan bakış açısıyla kod yazılması, güvenlik açıklarının henüz commit aşamasına gelmeden yakalanmasını sağlar. Bu durum “Security by Design” prensibinin en üst seviyede uygulanması anlamına gelir.
- **CI/CD’ye Otomatik Güvenlik Testlerinin Eklenmesi:** Kurum içi geliştirilen veya özelleştirilen güvenlik testleri pipeline’a entegre edilebilir.
- **Harici Pentest Öncesi Sistemin Olgunlaştırılması:** Harici pentest yaptırılmadan önce sistemin kurum içi testlerle olgunlaştırılması, harici pentest raporunun çok daha temiz çıkmasını sağlar. Bu durum hem maliyeti düşürmekte hem de prestij kazandırmaktadır.
- **Çoklu Uygulama Güvenliği:** Monorepo yapısındaki birden fazla uygulama ve servisin güvenlik testlerinin OSCP yetkinliği ile sürekli yapılabilmesi, geniş kapsamlı bir güvenlik değerlendirmesi sağlar.

OSCP’nin Tek Başına Yeterli Olmadığı Noktalar

- **Bağımsızlık Şartı:** NIS2 Direktifi, denetimlerin ve testlerin bağımsız bir tarafça gerçekleştirilmesini şart koşturmaktadır. Geliştirilen kodun, geliştiren ekip tarafından test edilmesi denetçi nezdinde objektif kabul edilmemektedir.
- **Akreditasyon Eksikliği:** OSCP kişisel bir sertifika olup, sertifika sahibini akredite bir “pentest firması” statüsüne taşımamaktadır. NIS2 kapsamında bazı ülke otoriteleri, pentest raporunun CREST, CHECK veya TIBER-EU gibi akreditasyonlara sahip bir kuruluş tarafından düzenlenmesini zorunlu tutmaktadır.
- **Kör Nokta Riski:** Geliştirici ekibin kendi koduna aşinalığı, özellikle geniş monorepo yapısında bazı güvenlik açıklarının “kör nokta” etkisiyle gözden kaçmasına neden olabilir. Harici bir göz, öngörülemeyen saldırı vektörlerini deneyebilir.
- **Raporun Yasal Geçerliliği:** NIS2 denetimi veya olası bir veri ihlali davasında, “Yazılımı geliştiren kişi aynı zamanda pentestini de gerçekleştirmiştir” ifadesi hukuki açıdan zayıf kalmaktadır. Bağımsız üçüncü taraf raporu mahkemede delil niteliği taşımaktadır.

Önerilen Hibrit Strateji

OSCP sertifikasının alınması ve pentest yetkinliğinin geliştirilmesi önemli bir yatırımdır. Bu yetkinliğin NIS2 stratejisine entegrasyonu için aşağıdaki hibrit model önerilmektedir:

Periyot	Faaliyet	Kapsam	Sorumlu
Her sprint sonu	Otomatik güvenlik taraması (SAST/DAST)	Tüm projeler	CI/CD (Otomatik)
Aylık	Manuel iç pentest (hızlı tur)	Kritik servisler ve uygulamalar	Kurum içi (OSCP sertifikalı personel)

6 ayda bir	Kapsamlı iç pentest ve raporlama	Tüm sistem bileşenleri	Kurum içi (OSCP sertifikalı personel)
Yılda bir	Bağımsız harici pentest ve rapor	Tüm sistem bileşenleri	Akredite firma
Büyük sürüm öncesi	Bağımsız harici pentest (hedefli)	Yeni eklenen bileşenler	Akredite firma

Bu strateji ile:

- NIS2'nin "bağımsızlık" şartı yıllık harici pentest ile karşılanır.
- Kurum içi sürekli testler "proaktif güvenlik kültürü" kanıtı olarak sunulur.
- Harici pentest raporları ile kurum içi test bulgularının örtüşmesi, iç süreçlerin etkinliğini göstermekte ve denetçi nezdinde olumlu karşılanmaktadır.
- OSCP sertifikası, SSDF belgesinde "Geliştirme ekibi OSCP sertifikalı güvenlik uzmanı içermektedir" ifadesiyle yer alarak kurumsal prestije katkı sağlar.

Denetçiye Sunulacak Anlatı

"GD-PMS geliştirme ekibimiz bünyesinde OSCP sertifikalı personel bulunmaktadır. Her ay tüm sistem bileşenlerimize (data-service, device-service, web-service, container-web, field, superadmin, desktop ve container-desktop) iç pentest uygulanmakta, tespit edilen bulgular derhal kapatılarak pipeline'ımıza yeni güvenlik testleri eklenmektedir. Ayrıca NIS2 bağımsızlık gerekliliği kapsamında yılda bir kez akredite bir firmadan harici pentest hizmeti alınmaktadır. İç pentest raporlarımız ve harici firma raporu ekte sunulmuştur. İki rapor arasındaki bulgu örtüşme oranının %90 seviyesinde olması, iç sürecimizin etkinliğini kanıtlamaktadır."

Bu anlatım, denetçiye ekibin konuyu ciddiyetle ele aldığı, hem yetkin hem şeffaf olduğu mesajını vermektedir.

Bu belge, NIS2 uyumluluk sürecinin yalnızca yazılım geliştirme ayağını kapsamaktadır. Canlı sistem izleme (SIEM), container güvenliği, fiziksel güvenlik ve tedarikçi yönetimi gibi diğer NIS2 alanları için ayrıca planlama yapılmalıdır.